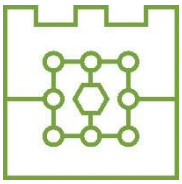


Politechnika Krakowska
im. Tadeusza Kościuszki

Wydział Informatyki i Matematyki



Wiktor Zmiendak

Numer albumu: 142706

Sprawozdanie

Projekt 4

Implementacja algorytmu SHA-256

Kraków 2026

Cel zadania

W ramach projektu należało przygotować własną implementację algorytmu SHA-256, sprawdzić jej poprawność oraz porównać jej działanie z biblioteką standardową Pythona. Dodatkowo trzeba było zbadać kilka podstawowych własności funkcji skrótu, czyli możliwość haszowania danych o różnej długości, efekt lawiny po zmianie jednego bitu, odległość Hamminga dla dwóch różnych tekstów, stałą długość otrzymywanego skrótu oraz czas generowania skrótu w porównaniu z funkcją biblioteczną.

Implementacja

Sama implementacja opiera się na klasycznym schemacie SHA-256. Na początku ustawiane jest osiem 32-bitowych rejestrów startowych. Następnie wiadomość jest dopełniana w taki sposób, aby jej długość była zgodna z wymaganiami algorytmu, czyli aby można było ją podzielić na bloki po 512 bitów, a następnie każdy blok jest przetwarzany osobno.

Pierwsze szesnaście słów harmonogramu pobieranych jest bezpośrednio z danych wejściowych, a kolejne słowa są wyprowadzane z poprzednich przy użyciu rotacji bitowych i operacji logicznych. Później wykonywanych jest 64 rundy kompresji, w których aktualizowane są rejestry robocze. Na końcu wyniki wszystkich bloków są łączone w jeden stan i zamieniane na zapis heksadecymalny.

Testowanie

Poprawność rozwiązania zweryfikowano wieloetapowo. Najpierw porównano skróty z własnej implementacji z wynikami hashlib.sha256() dla i uzyskano pełną zgodność. Następnie sprawdzono własności algorytmu tzn czy program poprawnie haszuje dane o różnych długościach oraz bardzo długie wiadomości.

W teście efektu lawiny zmiana jednego bajtu w danych wejściowych spowodowała różnicę 121 z 256 bitów skrótu, co potwierdza wysoką czułość funkcji na minimalne zmiany. Dla dwóch podobnych tekstów, Ala ma kota i Ala ma psa, odległość Hamminga wyniosła 132/256 bitów, czyli ponad połowę długości hashu. Otrzymane wyniki są zgodne z oczekiwaniami dla SHA-256.

W części wydajnościowej porównano czas działania własnej implementacji i biblioteki standardowej gdzie otrzymano odpowiednio 0.003443 s oraz 0.000022 s. Oznacza to, że hashlib działa około 159 razy szybciej, co jest naturalne, ponieważ korzysta z silnie zoptymalizowanej implementacji w języku C.

Wyniki

```
================================================
Poprawność: SHA-256 zgadza się z hashlib

[1] Czy można skrócić tekst dowolnej długości?
Wszystkie długości obsługiwane

[2] Zmiana 1 bitu → roznica skrotu (efekt lawiny)
Zmieniony bajt #40 → 121/256 bitów różnicy w hashu

[3] Odleglosc Hamminga dla różnych tekstów
Ala ma kota' vs 'Ala ma psa' → 132/256 bitów różnicy

[4] Czy hash ma zawsze tę samą długość?
TAK - zawsze 64 znaków (= 256 bitów)

[5] Porównanie czasu: custom vs hashlib
Custom:  0.003443s
Hashlib: 0.000022s
Hashlib jest szybsza ~159x (oczekiwane - C vs Python)
```